

MAE 3780 Robot Project Final Report

Group Number: 58

Members: Alison Chavez (avc35), Ana Badea (amb675)

1. Robot Design and Strategy Overview.

We aimed to be simple in our design and strategy. Our strategy was to open ourselves up to being as wide as possible with a 3D printed claw, then to quickly collect in a simple path from one side of the middle to the other. At the start of the match, we wrote our code to open our claw, move forward until the left black border was detected, turn until the opposite color was detected, and then turn again. This allowed our robot to follow a path between the intersection of the blue and yellow section, until the black border was detected, where we would then turn right to go back to our original color section.

For our robot, we primarily designed an L-shaped claw that would open us to the maximum legal dimensions upon start. Our claw was composed of two arms and symmetric on either side, with one long thin arm being mounted on one positional servo placed in front of our wheels. We utilized the color sensor to detect blue and yellow and two QTI sensors to detect black at the left or right side of our robot. We bought and utilized the bigger wheels from the lab to increase our speed.

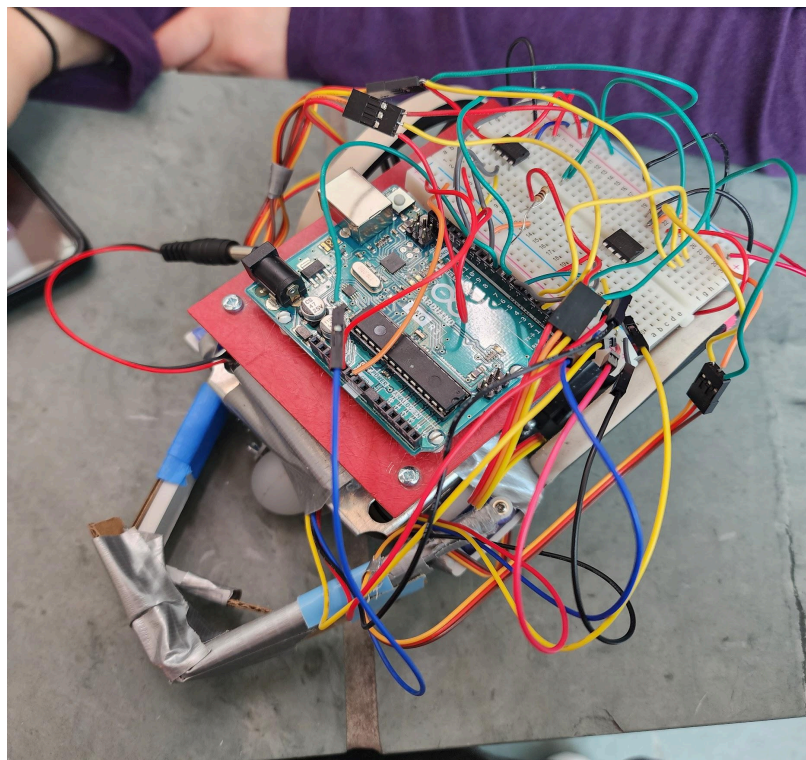


Figure 1. Final Robot from the top

2. Design Process Reflection.

Our team decided to design our robot following the specification with the milestones. We tended to procrastinate in our design process, such as starting a milestone the week it was due. Oftentimes, we would run into issues with our code not working, which would then delay progress and led to us completing the milestones near or on the day it was due. We think we should have CADed sooner and spent more time iterating our mechanical design and code. Many of our problems stemmed from rushed or a lack of testing (questionable color detection, a very simple code strategy, and very very flimsy claws). This led to us making last minute changes, such as taping cardboard onto our claws to strengthen them, which limited our time to test fully.

3. Competition Analysis.

We did not do that well during competition and lost all our matches. Our strategy was very common which meant oftentimes our robot would interact with another robot in the block collection section. Our claws were very flimsy and would get frequently caught on the opposing team's bots. This would lead to us losing the blocks we managed to collect, since they were either pushed away or ended up in the other robot's perimeter. Because our design was flimsy and our bot was light we also frequently got pushed off the mat or our claw folded back to an unusable position.

A strength was our wide claw, which collect many blocks at a single time, but in contrast the flimsy structure of it was also our primary weakness. Other weaknesses was our color detection and turning, since sometimes the blue floor would be mistaken as black or the rotation to turn 90 degrees would veer to wide, and therefore minimizing our time in the block collection section.

4. Conclusions.

Overall, our team enjoyed this experience of being able to design a robot from start to finish. We enjoyed being able to brainstorm and strategize together to create our robot design, and were able to accomplish all aspects of the milestones and project together since we collaborated for all mechanical, electrical, and software aspects. While we wish our robot could have performed better, we also recognize that as a team of 2 it was difficult to allocate time specifically to testing and iteration since our primary focuses throughout the week was to accomplish the milestones.

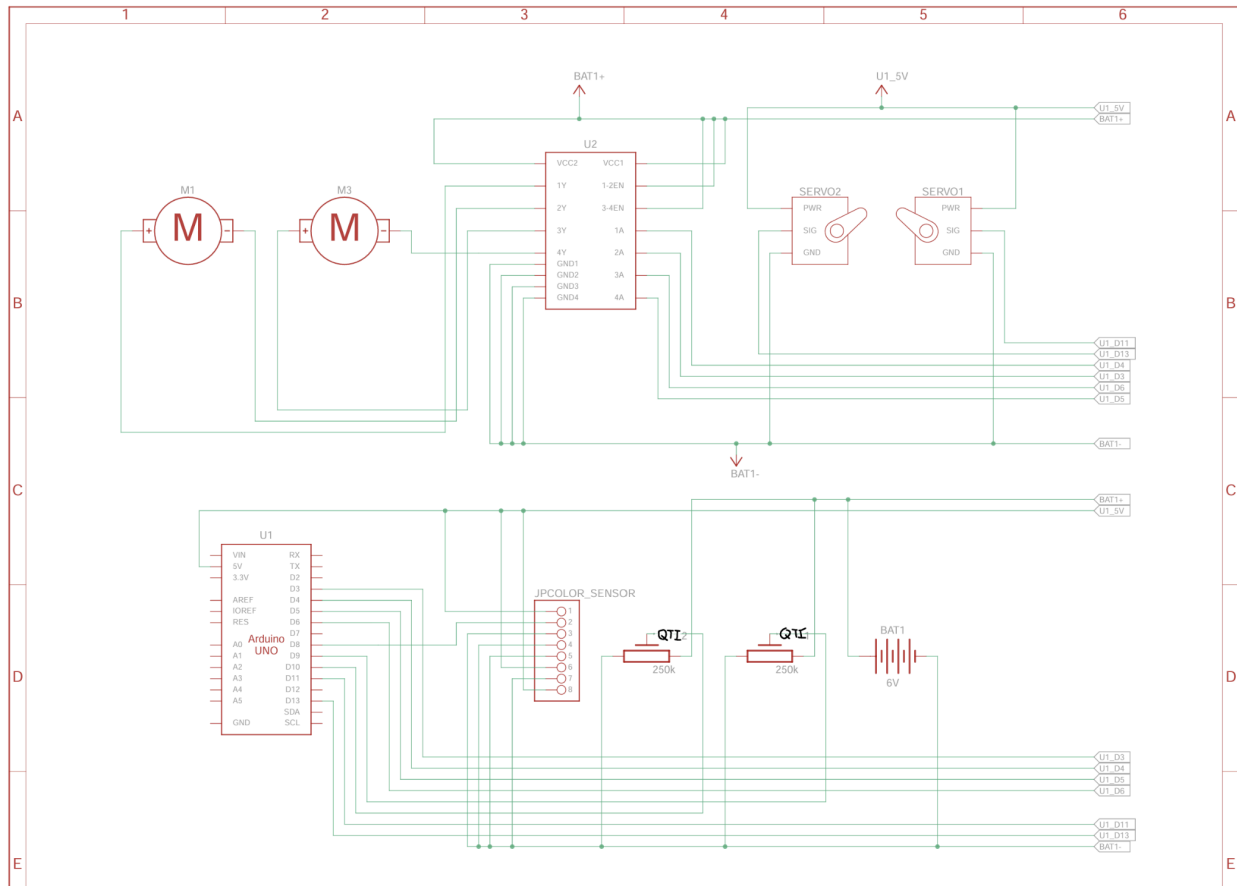
If we could do the project over again, we would recommend starting early with CAD and writing the code for your strategy. We only began CAD for milestone 4, which was too late in our opinion since the turn around from obtaining the RPL parts and implementing our claws into our CAD was very slim. Had we started earlier, we would have had more time to potentially redesign our claws and improve our code. In addition, we advise future students to not procrastinate and start the milestones as early as possible. We often got stressed when our code or claw were not working the day the

milestone was due, and saw a lot of other teams in a similar dilemma. Avoiding these situations would help mitigate stress and also ensure a fun time during the robot project!

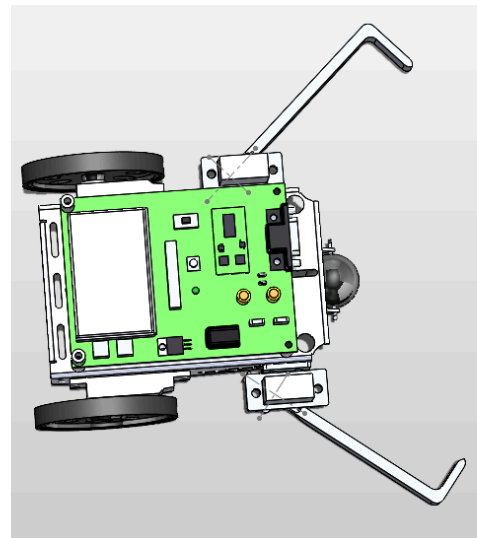
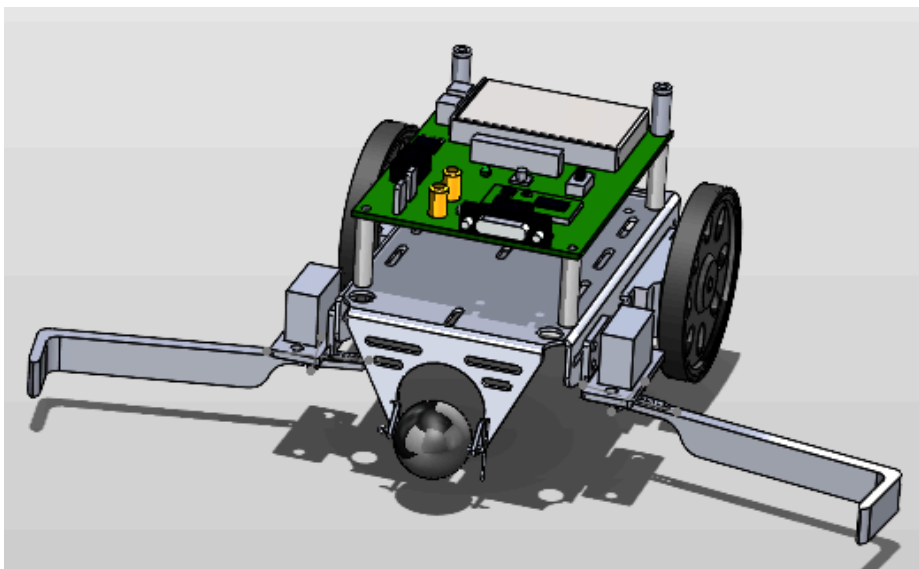
Appendix A: bill of materials (BOM). Your bill of materials only needs to include additional parts that your group purchased and included in your budget, you do not need to include parts from your lab kit. Be sure to also include any purchased parts that did not end up being used in the final design (3D prints, lasercut components, etc.).

Part Name	Type of part	Quantity	Individual Cost	Total Cost
Larger Diameter Wheels	Purchased	2	2	4
Positional Servos	Purchased	1	3	3
Servo Mounts	3-d printed	2	1.66	3.32
Claws	3d printed	2	3.60	7.20
Total Cost				17.52

Appendix B: Circuit diagram made with TinkerCAD.



Appendix C: CAD files and drawings. Include screenshots of relevant CAD files or drawings used for manufacturing individual parts. Also include screenshots of part volumes and weight calculations (or photos of the parts being weighed) corresponding to the lines in your BOM.

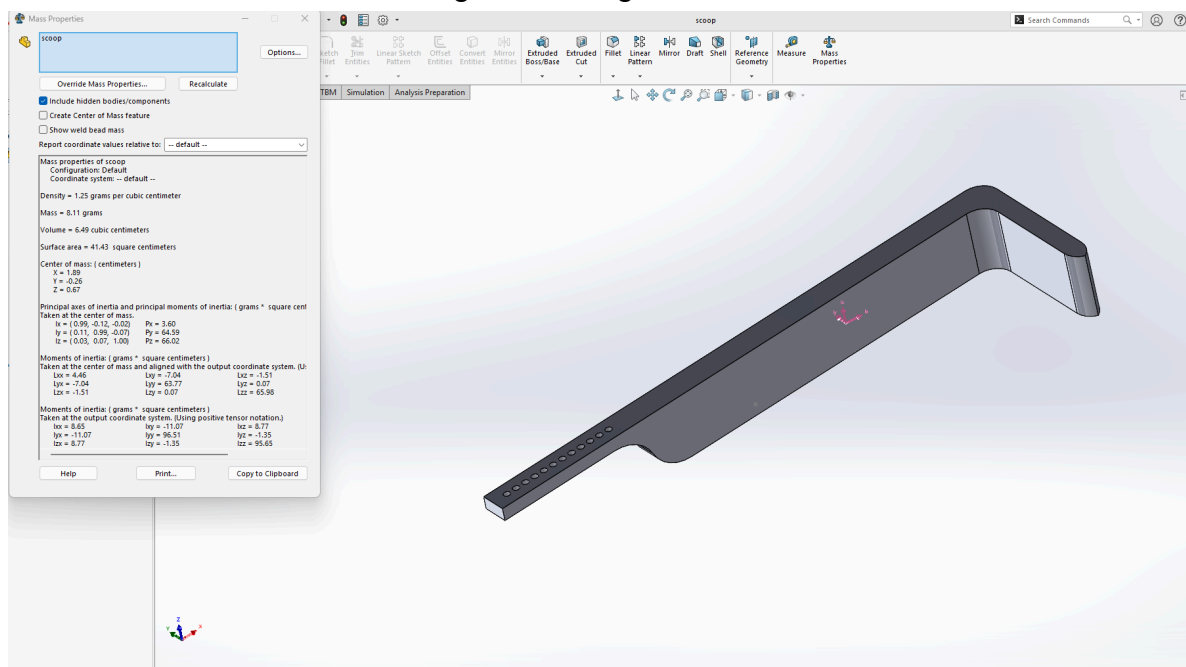


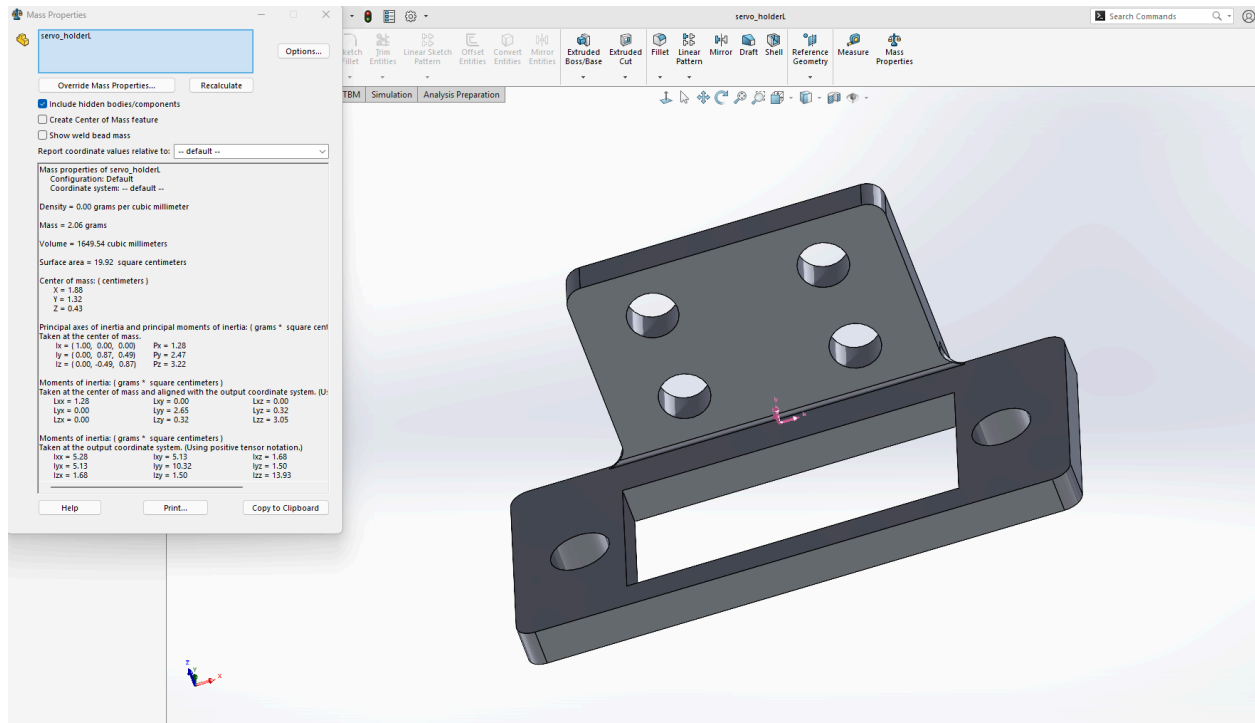
Our CAD:

We printed a total of 4 parts, two identical servo mounts and two mirrored but otherwise identical scoops. For our weight/printing cost calcs we found the volumes the parts (shown below to do the calculations

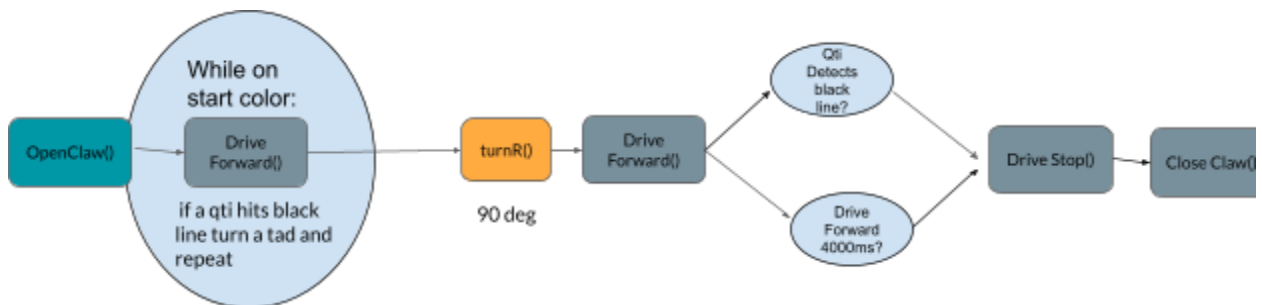
The Scoops: $\$1 + \$0.40/\text{g} * 6.49\text{g} = \3.60 each

The Servo Mounts: $\$1 + \$0.40/\text{g} * 1.64954\text{g} = \1.66 each





Appendix D: Flowchart. Your robot's strategy/algorithm represented in flowchart form. The flowchart does not need to represent every single line of code, but it should show high-level behaviors and functions.



Appendix E: Code: commented Arduino code. A clear understanding of each line of code should be demonstrated. References must be provided if used. Referenced: Tinkercad tutorials for positional servos, other motors, lab material for Color sensor and class material for QTI

CODE:

```
#include <avr/io.h>
```

```

#include <avr/interrupt.h>
#include <util/delay.h> // Essential for _delay_ms()

// Global variables for sensor timing
volatile uint16_t timer_val = 0;
int dt = 100; // delay time for loop
int maxpwm = 19; // maximum value for OCR2B register
int minpwm = 8; // minimum value for OCR2B register

//--- ISR: Reads the timer value to determine frequency/period
ISR(PCINT0_vect) {
    if (PINB & (1 << PB0)) {
        TCNT1 = 0; // Reset timer on Rising Edge
    } else {
        timer_val = TCNT1; // Store count on Falling Edge
    }
}

//--- Initialization: Pins and Timers
void initRobot() {
    // Color Sensor Pin: PB0 as input
    DDRB &= ~(1 << PB0);
    PCICR |= (1 << PCIE0); // Enable Pin Change Interrupt 0

    // Motor Pins: PD5, PD6 and PB1, PB3 as outputs
    DDRB |= (1 << PB1) | (1 << PB3);
    DDRD |= (1 << PD5) | (1 << PD6);

    // QTI Sensor Pins: PD2 and PD3 as outputs (initially)
    DDRD |= (1 << PD2) | (1 << PD7); //changed to PD7 from PD3

    //Micro servo pins: PD3 controls both
    DDRD |= (1 << PD3);

    // Timer1: Normal mode, Prescaler 1
    TCCR1A = 0;
    TCCR1B = (1 << CS10);
    // Fast PWM, clear on compare
    TCCR2A = 0b00100011;
    TCCR2B = 0b00001100; // prescaler 64

```

```

OCR2A = 249;    // ~20ms period (servo standard)
OCR2B = minpwm; //test maybe exclude
TCCR2A = 0b00100001;
TCCR2B = 0b00001111;
sei();         // Enable global interrupts
}

//--- getColor: Period calculation
int getColor() {
    PCMSK0 |= (1 << PCINT0); // Enable interrupt for PB0
    _delay_ms(10);
    PCMSK0 &= ~(1 << PCINT0); // Disable to avoid noise during processing

    // Math: timer * (1/clock_speed). Assuming 16MHz: 0.0625us per tick.
    // Period = 2 * timer_val * 0.0625
    return (int)(2 * timer_val * 0.0625);
}

//--- Movement Functions
void drive_forward() {
    PORTD = (PORTD & ~((1 << PD5) | (1 << PD6))) | (1 << PD5);
    PORTB = (PORTB & ~((1 << PB1) | (1 << PB3))) | (1 << PB3);
}

void drive_backward() {
    PORTD = (PORTD & ~((1 << PD5) | (1 << PD6))) | (1 << PD6);
    PORTB = (PORTB & ~((1 << PB1) | (1 << PB3))) | (1 << PB1);
}

void drive_L() {
    PORTD = (PORTD & ~((1 << PD5) | (1 << PD6))) | (1 << PD6);
    PORTB = (PORTB & ~((1 << PB1) | (1 << PB3))) | (1 << PB3);
}

void drive_R() {
    PORTD = (PORTD & ~((1 << PD5) | (1 << PD6))) | (1 << PD5);
    PORTB = (PORTB & ~((1 << PB1) | (1 << PB3))) | (1 << PB1);
}

```



```

void drive_stop() {
  PORTD &= ~((1 << PD5) | (1 << PD6));
  PORTB &= ~((1 << PB1) | (1 << PB3));
}

void claw_stop() {
  OCR2B = 14; //set stop position like partially open
}

void claw_open() { //fully open
  OCR2B = 19;
  Serial.print("open claw:");
  Serial.println(OCR2B);
}

void claw_close() { //retract
  Serial.println("close claw");
  while (OCR2B > 12) { // loop in one direction
    OCR2B--;          // increase OCR2B
    Serial.println(OCR2B);
    _delay_ms(dt);
  }
  OCR2B = 13; // adjust this too
}

void claw_start() { //to fit in box

  Serial.println("claw start");
  OCR2B = 8;
}
//determining our color
int determineState(int p) {
  Serial.print("color:");
  Serial.println(p);
  if (p < 210) return 2; // Yellow
  if (p >= 210) return 1; // Blue
  return 3;             // Other
}

//--- QTI Sensor Reading

```

```

uint16_t readQTI(uint8_t pin) {
    uint16_t time = 0;
    DDRD |= (1 << pin); // Set as output
    PORTD |= (1 << pin); // Drive HIGH
    _delay_us(10);      // Charge

    DDRD &= ~(1 << pin); // Set as input
    PORTD &= ~(1 << pin); // Disable pull-up

    while (PIND & (1 << pin)) {
        time++;
        _delay_us(1);
        if (time > 2000) break;
    }

    return time;
}

void test_servo_reverse() { //this was to find servo positioning not used in comp
/*
    for(int i =3; i <= 4; i++){
        OCR2B = i;
        Serial.println(i);
        _delay_ms(100);
    }
*/
//OCR2B++;
//OCR2B++;
TCCR2A = 0b00100001;
TCCR2B = 0b00001111;
OCR2A = 156;

// OCR2B register to move the servo
OCR2B = minpwm;
while (OCR2B < 10) { // loop in one direction
    OCR2B++;          // increase OCR2B
    Serial.println(OCR2B);
    _delay_ms(1000);
}
}

```

```

//MAIN SEQUENCE START
int main(void) {
    initRobot();
    Serial.begin(9600);
    int halfturn = 1000;
    //_delay_ms(2000);
    int startColor = determineState(getColor());
    _delay_ms(200);
    claw_open();
    while (1) {
        //claw_close();
        //drive_forward();
        claw_open();
        _delay_ms(100);
        //claw_close();
        //_delay_ms(1000);

        //claw_stop();
        //Serial.println("claw stop");
        //_delay_ms(000);

        uint16_t qti_right = readQTI(PD2);
        uint16_t qti_left = readQTI(PD3);
        Serial.print("qti_right");
        Serial.println(qti_right);
        Serial.print("qti_left");
        Serial.println(qti_left);
        //open claw

        //claw_stop();
        //until left corner
        //detect edge of board
        while (qti_left < 650 && qti_right < 650) { //not black
            drive_forward();
            _delay_ms(50);
            drive_stop();
            qti_right = readQTI(PD2);
            qti_left = readQTI(PD3);
        }
    }
}

```

```

//turn
drive_R();
_delay_ms(halfturn / 2);
drive_stop();

//until opp color
while (determineState(getColor()) == startColor) {
  if (qti_left > 650) {
    drive_R();
    _delay_ms(30);
    drive_stop();
  }
  else if (qti_right > 650) { //if hit black turn to stay on board
    drive_L();
    _delay_ms(30);
    drive_stop();
  }
  drive_forward();
  _delay_ms(50);
  drive_stop();
  qti_right = readQTI(PD2);
  qti_left = readQTI(PD3);
}
//turn
drive_R();
_delay_ms(halfturn / 2);
drive_stop();
//drive until opp edge
qti_right = readQTI(PD2);
qti_left = readQTI(PD3);
int tb = 0;
while (qti_left < 650 && qti_right < 650) { //not black
  drive_forward();
  tb = tb + 50;
  _delay_ms(50);
  drive_stop();
  qti_right = readQTI(PD2);
  qti_left = readQTI(PD3);
  Serial.println(tb);
  if (tb == 4000) { //in case qti doesnt detect black stop so we don't drive off board

```

```

Serial.println("TB STOP");
_delay_ms(100);

claw_stop();
_delay_ms(100);
claw_stop();    // turn off servo

while (1);      // stay here until the match ends
}
}
drive_stop();
_delay_ms(100);
claw_stop();
_delay_ms(100);

while (1);      // 4. Stay here until the match ends  /*

/*
drive_forward();
_delay_ms(1470*2);
drive_stop();
_delay_ms(1000000);
//turn
*/

/*
drive_R();
_delay_ms(halfturn / 2);
drive_stop();

while (qti_left < 650 && qti_right < 650) { //not black
drive_forward();
_delay_ms(50);
drive_stop();
qti_right = readQTI(PD2);
qti_left = readQTI(PD3);
}

if (qti_left > 650 && qti_right > 650) { //on black
drive_stop();

```

```
while (1)
    ;
}
    claw_close();
    claw_stop();
    */
}
}
```

References include TinkerCAD tutorials for the positional servo and class lecture materials.